

M114 Thema-1: DATEN CODIEREN

- Massvorsätze (*Repetition*)
 - Bit und Byte (*Repetition*)
 - Zahlensysteme BIN/OCT/DEZ/HEX (*Repetition*)
 - AND/OR/NOT/XOR (*Repetition*)
 - Binär addieren, negative Binärzahlen mit 2-er Komplement, DataOverflow
 - Fließkommazahlenformat
 - Modulo-Funktion
-
- Parallele/Serielle Datenübertragung
 - Data-Clock, Taktsignal bei der Datenübertragung
 - Online-HEX-Editor hexed.it, Notepad++
-
- Alphanumerische Codes ASCII, ANSI-ASCII, ISO8859-x
 - Unicode, Unicode V2, UTF8, UTF16 LE-BOM und RE-BOM (Byte-Order)
-
- Optional: Barcodes EAN8 und EAN13, QR-Codes.

SI-Präfixe (Dezimalpräfixe)

[T] → Tera → 10^{12} → 1'000'000'000'000 → Billion

[G] → Giga → 10^9 → 1'000'000'000 → Milliarde

[M] → Mega → 10^6 → 1'000'000 → Million

[k] → kilo → 10^3 → 1'000 → Tausend

IEC-Präfixe (Binärpräfixe)

[Ti] → Tebi → 2^{40} → 1'099'511'627'776

[Gi] → Gibi → 2^{30} → 1'073'741'824

[Mi] → Mebi → 2^{20} → 1'048'576

[Ki] → Kibi → 2^{10} → 1'024

SI-Präfixe z.B. bei Geschwindigkeitsangaben wie 1Mb/sec

IEC-Präfixe bei Speichergrößen wie z.B. 1MiB

BTW: B = Byte

b = bit

1B = 8b

16b = 1 Word

LSB = Least Significant Bit / Kleinstwertigstes Bit

MSB = Most Significant Bit / Höchstwertigste Bit

(Wichtige Angabe bei z.B. bei Parallelverbindungen damit Stecker nicht falsch angeschlossen wird)

Der Computer versteht nur 0 und 1

(Wenigstens solange die Quantencomputer mit ihren Q-Bits nicht grossflächig Einzug gehalten haben.)

- Verarbeitung, Speicherung Binär (BIN)
- Darstellung zur besseren Lesbarkeit/Übericht in Hexadezimal (HEX)

Für sie als Programmierer/in interessant:

- Wie und wo wird eine Zahl gespeichert?
- Welche Basis-Datentypen gibt's es?
- Was sind die Wertebereiche dieser Datentypen?
- Was passiert bei einer Überschreitung eines Wertebereichs?

Für sie als Informatiker interessant:

- Warum ist der Speicherplatz immer ein Vielfaches von 2?
- Wie kann ich die Bits und Bytes sichtbar machen?

(Einsatz von HEX-Editor. Empfehlung: hexed.it)

Für Informatiker wichtige Zahlensysteme:

BIN **Binärsystem**, Zweiersystem, Dualsystem

Basis: 2

Zeichenvorrat: 0, 1

Zahlenbeispiel: $0110'1011_B$

DEZ **Dezimalsystem**, Zehnersystem

Basis: 10

Zeichenvorrat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Zahlenbeispiel: 2025_D

HEX **Hexadezimalsystem**, Sechzehnersystem

Basis: 16

Zeichenvorrat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (=10), B (=11), C (=12), D (=13), E (=14), F (=15)

Zahlenbeispiel: $FF3C_H$ oder $0xFF3C$

Hinweis: Eine Hex-Ziffer entspricht einer vierstelligen Binärzahl bzw. 4 Bit!

Für Informatiker wichtige Zahlensysteme:

BIN **Binärsystem**, Zweiersystem, Dualsystem

Basis: 2

Zeichenvorrat: 0, 1

Zahlenbeispiel: $0110'1011_B$

Beispiel für die Umrechnung von Binär 1011 in Dezimal:

2^3	2^2	2^1	2^0	Wertigkeit der Binärstelle (Basis=2)
8	4	2	1	Wertigkeit Dezimal
1	0	1	1	Binärwert
8	0	2	1	Die SUMME ergibt das Dezimaläquivalent: 11

Somit: $1011_B = 11_D$

Für Informatiker wichtige Zahlensysteme:

BIN **Binärsystem**, Zweiersystem, Dualsystem

Basis: 2

Zeichenvorrat: 0, 1

Zahlenbeispiel: $0110'1011_B$

Beispiel für die Umrechnung Dezimal 34 in Binär: (Fortlaufende Division)

34	DIV 2 =	17	REST 0
17	DIV 2 =	8	REST 1
8	DIV 2 =	4	REST 0
4	DIV 2 =	2	REST 0
2	DIV 2 =	1	REST 0
1	DIV 2 =	0	REST 1



$100010_B = 34_D$

Für Informatiker wichtige Zahlensysteme:

HEX **Hexadezimalsystem**, Sechzehnersystem

Basis: 16

Zeichenvorrat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (=10), B (=11), C (=12), D (=13), E (=14), F (=15)

Zahlenbeispiel: FF3C_H oder 0xFF3C

Hinweis: Eine Hex-Ziffer entspricht einer vierstelligen Binärzahl bzw. 4 Bit!

Beispiel für die Umrechnung Hexadezimal 5B7 in Binär:

4 Bit entsprechen einer HEX-Ziffer: 0000=0 bis 1111=F

5	B	7	HEX-Wert
8421	8421	8421	Wertigkeit
0101	1011	0111	Hex-Ziffer in Binär

Somit entspricht der HEX-Wert 5B7_H dem Binärwert 0101'1011'0111_B

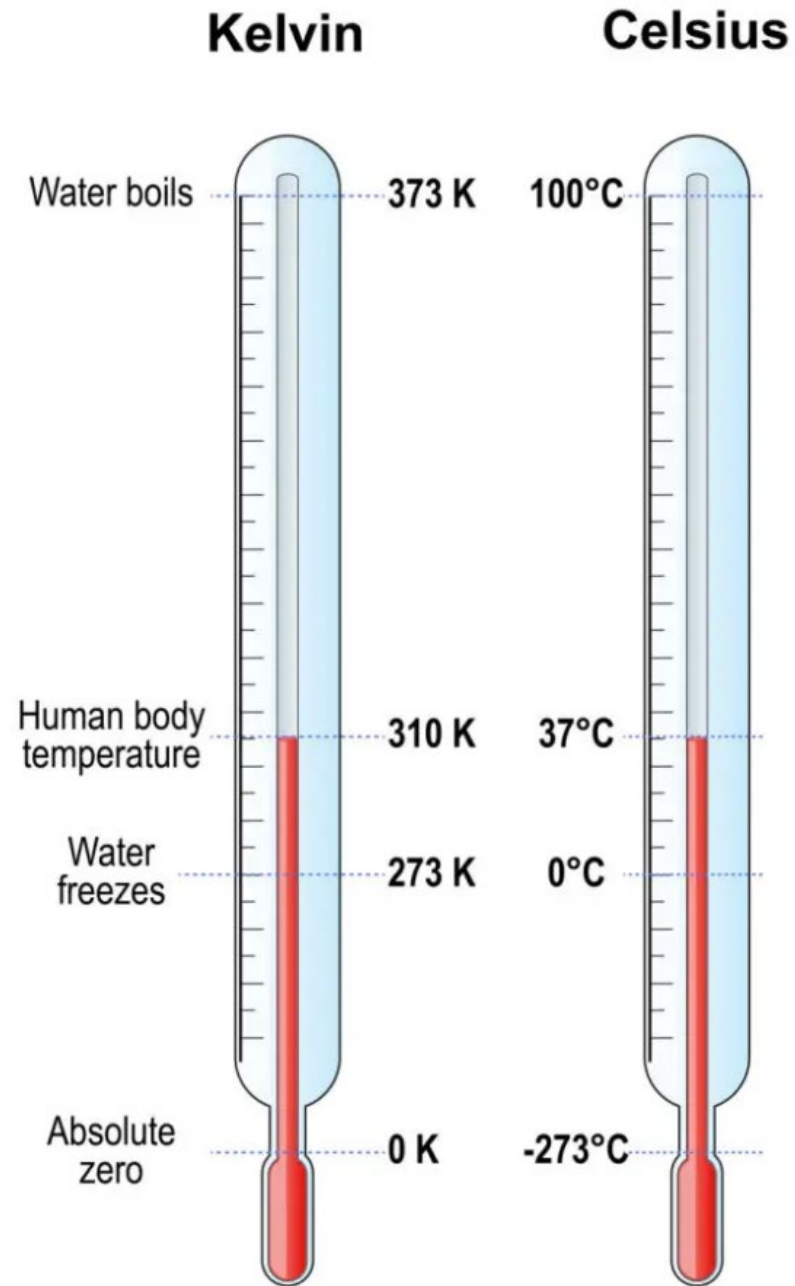
HEX **Hexadezimalsystem**, Sechzehnersystem

Zeichenvorrat: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (=10), B (=11), C (=12), D (=13), E (=14), F (=15)

8	4	2	1	HEX
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	A
1	0	1	1	B
1	1	0	0	C
1	1	0	1	D
1	1	1	0	E
1	1	1	1	F

- 4 Bit entsprechen einer HEX-Ziffer
- Die "Geschwindigkeit" des Bit-Wechsels 0 zu 1 oder 1 zu 0 halbiert sich bei jeder weiteren linken Kolonne
- Das Bit ganz rechts nennt man LSB (Least Significant Bit)
- Das Bit ganz links nennt man MSB (Most Significant Bit)

Binärzahlen mit/ohne Vorzeichen



Negative Zahlen = Offset

Binärzahlen mit/ohne Vorzeichen

0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10/A
1011	11/B
1100	12/C
1101	13/D
1110	14/E
1111	15/F

unsigned integer 4Bit
Wertebereich: 0..15

Binärzahlen mit/ohne Vorzeichen

0111	+7
0110	+6
0101	+5
0100	+4
0011	+3
0010	+2
0001	+1
0000	0
1001	-1
1010	-2
1011	-3
1100	-4
1101	-5
1110	-6
1111	-7

Vorderstes Bit als Vorzeichen?

Funktionieren negative Zahlen so?

Was ist mit **1000** ? Gibt's es nicht?

Wie wärs mit einer Proberechnung?

0001 + (Entspricht +1)

1010 (Entspricht -2)

1011 (Entspricht -3)

Na, also!

Binärzahlen mit/ohne Vorzeichen

0111	+7
0110	+6
0101	+5
0100	+4
0011	+3
0010	+2
0001	+1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Gegeben: signed integer 4Bit
Mit Wertebereich: -8..+7

2-er Komplement-Rezept:

1. Ich möchte z.B. eine -5
2. Ich nehme eine +5 (=0101)
3. Ich negiere jede Stelle einzeln
Aus 0101 wird 1010
4. Ich addiere 1 dazu
 $1010 + 1 = 1011$
1011 ist -5. Wers nicht glaubt, siehe Tabelle

Binärzahlen mit/ohne Vorzeichen

0111	+7
0110	+6
0101	+5
0100	+4
0011	+3
0010	+2
0001	+1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Gegeben: signed integer 4Bit

Mit Wertebereich: -8..+7

2-er Komplement-Rezept:

Alternative Variante mit Wertigkeit:

-8	+4	+2	+1
1	0	1	1

Man beachte die negative Wertigkeit der höchsten Stelle (MSB=Most significantest bit)

Fliesskommazahlen (Gleitkomma)

INTEGER HAT ANDEREN ZWECK ALS FLOAT

Dies wäre falsch: (Pseudocode)

```
float f = 300'000'000.0;  
f = f + 1.0; //liegt nicht mehr in der Genauigkeit  
printf("%f",f); //Ergibt immer noch 300'000'000
```

Dies ist ok: (Pseudocode)

```
float f = 3.0;  
f = 1.0 / f;
```

FLOATing point number = Fliesskommazahl

Beispiel:

Die Zahl Pi kann man so schreiben:

+3.14156

+3.14156 x 10⁺⁰

+314.156 x 10⁻³

+0.0031456 x 10⁺³

Mantisse=Zahlen und Vorzeichen vor dem Exponenten

Basis=10

Exponent=Zahlen und Vorzeichen im Exponenten

Norm IEEE 754

32 Bit → Single Precision

64 Bit → Double Precision

Gleitkommazahl: $x = v * m * b^e$

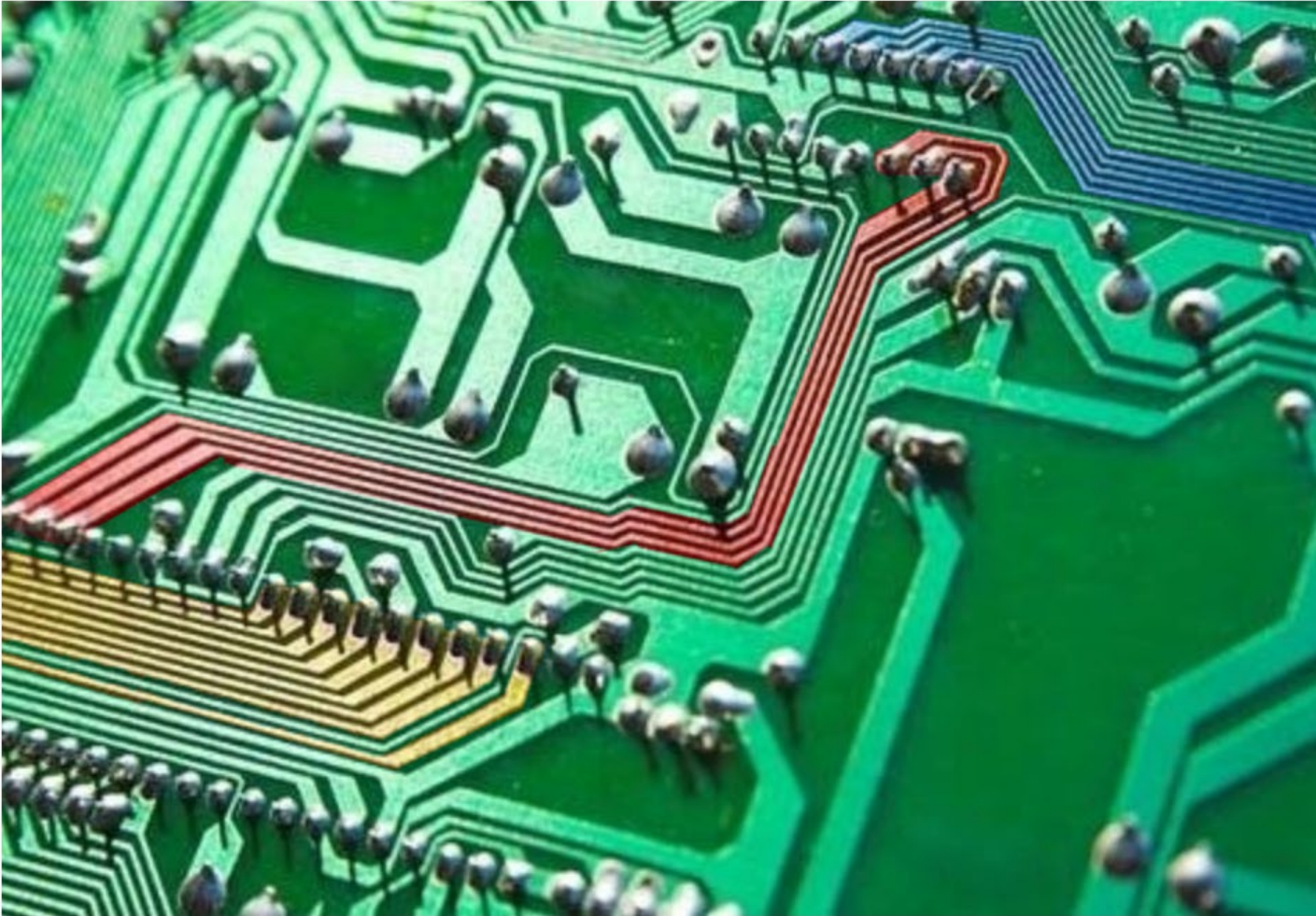
v: Vorzeichen → 1 Bit

m: Mantisse → Single: 23 Bit, Double: 52 Bit

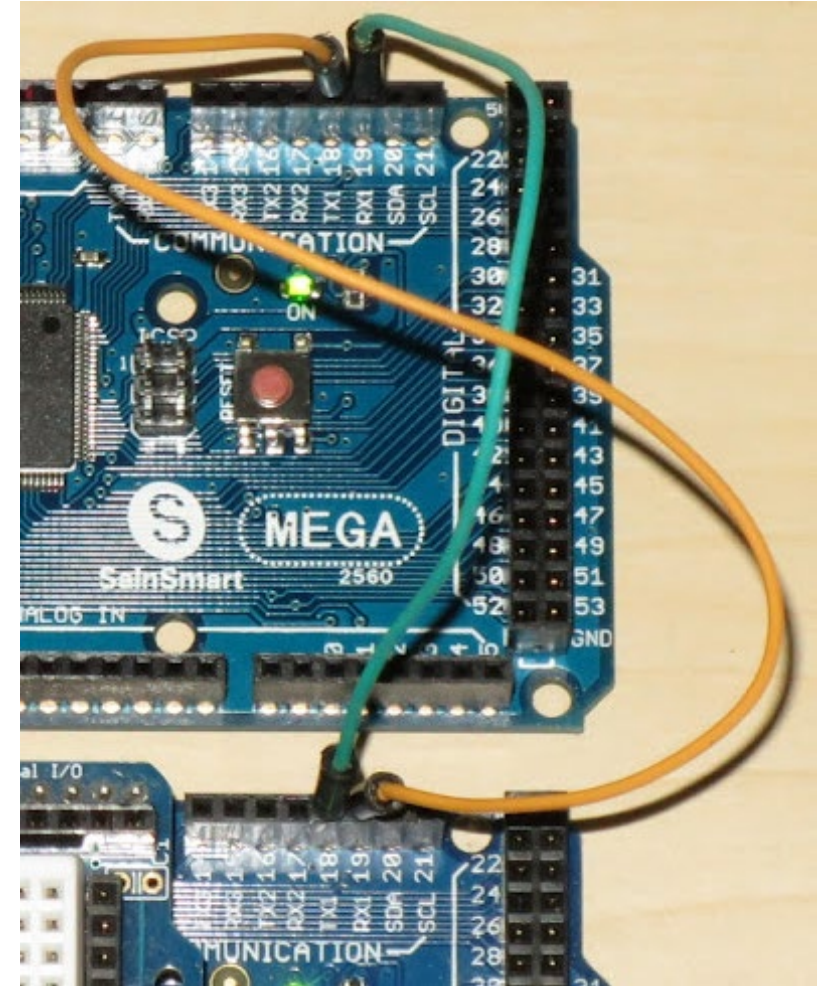
b: Basis → Bei normalisierten Gleitkommazahlen ist $b=2$

e: Exponent → Single: 8 Bit, Double: 11 Bit

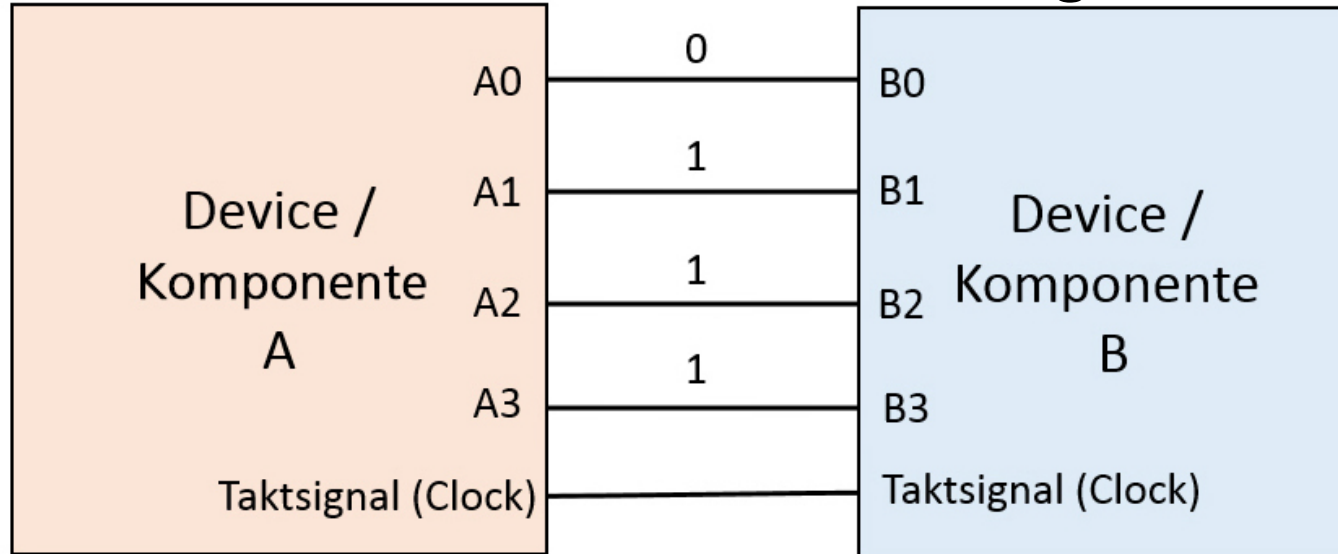
Parallele Datenverbindung



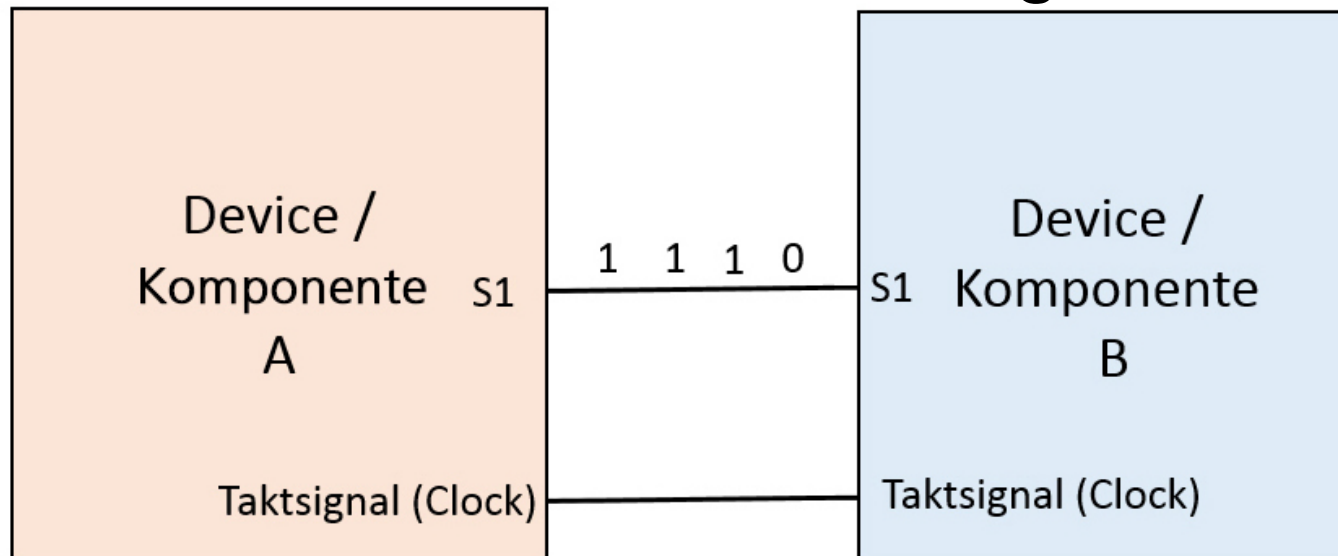
Serielle Datenverbindung



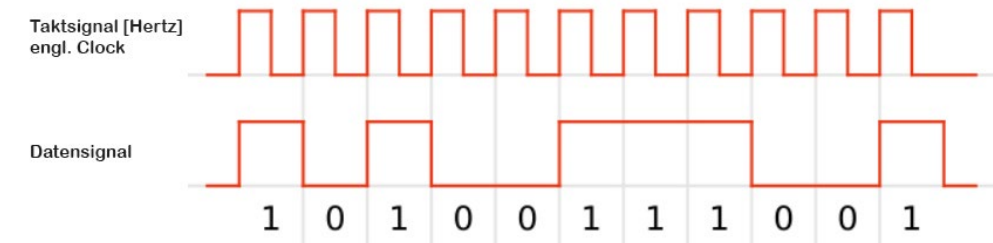
Parallele Datenverbindung



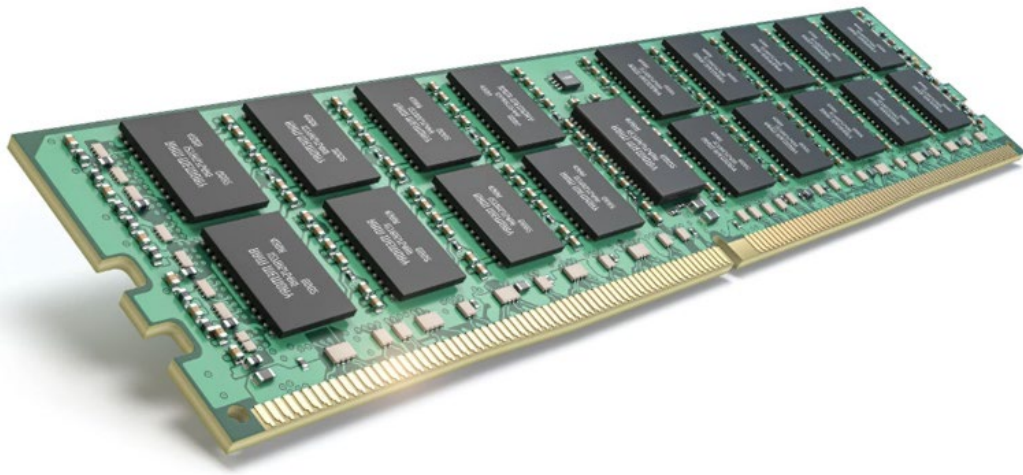
Serielle Datenverbindung



Taktsignal (Rechtecksignal)



Primärspeicher (Flüchtig / RAM)



Adressbus 4 Bit

0..15 (16 Speicherstellen)

1111

0010

0001

0000

Adresse

0	1	0	1	1	1	1	0
1	0	0	1	1	0	1	0
0	1	0	1	1	0	0	0
1	0	0	1	1	0	1	0

Kapazität des Speichers:
16 x 8 bit = 128 bit
oder
16 x 1 Byte = 16 Byte

Datenbus 8 Bit

1 Byte pro
Speicherstelle

*Dies ist der Grund dafür,
Warum Speichergrößen mit
Binärpräfixen (kiB, MiB etc.)
angegeben werden.*

Codierung von Text



DEC	HEX	BIN	CHAR	BEZEICHNUNG
000	00	00000000	NUL	Null Character
001	01	00000001	SOH	Start of Header
002	02	00000010	STX	Start of Text
003	03	00000011	ETX	End of Text
004	04	00000100	EOT	End of Transmission
005	05	00000101	ENQ	Enquiry
006	06	00000110	ACK	Acknowledgment
007	07	00000111	BEL	Bell
008	08	00001000	BS	Backspace
009	09	00001001	HT	Horizontal Tab
010	0A	00001010	LF	Line Feed
011	0B	00001011	VT	Vertical Tab
012	0C	00001100	FF	Form Feed
013	0D	00001101	CR	Carriage Return
014	0E	00001110	SO	Shift Out
015	0F	00001111	SI	Shift In
016	10	00010000	DLE	Data Link Escape
017	11	00010001	DC1	XON Device Control 1
018	12	00010010	DC2	Device Control 2
019	13	00010011	DC3	XOFF Device Control 3
020	14	00010100	DC4	Device Control 4
021	15	00010101	NAK	Negative Acknowledgement
022	16	00010110	SYN	Synchronous Idle
023	17	00010111	ETB	End of Transmission Bloc
024	18	00011000	CAN	Cancel
025	19	00011001	EM	End of Medium
026	1A	00011010	SUB	Substitute
027	1B	00011011	ESC	Escape
028	1C	00011100	FS	File Separator
029	1D	00011101	GS	Group Separator
030	1E	00011110	RS	Request to Send Record Separator
031	1F	00011111	US	Unit Separator

DEC	HEX	BIN	CHAR	BEZEICHNUNG
032	20	00100000	SP	Space
033	21	00100001	!	Exclamation mark
034	22	00100010	"	Double quote
035	23	00100011	#	Number sign
036	24	00100100	\$	Dollar sign
037	25	00100101	%	Percent
038	26	00100110	&	Ampersand
039	27	00100111	'	Single quote
040	28	00101000	(	Left opening parenthesis
041	29	00101001	)	Right closing parenthesis
042	2A	00101010	*	Asterisk
043	2B	00101011	+	Plus
044	2C	00101100	,	Comma
045	2D	00101101	-	Minus or dash
046	2E	00101110	.	Dot
047	2F	00101111	/	Forward slash
048	30	00110000	0	
049	31	00110001	1	
050	32	00110010	2	
051	33	00110011	3	
052	34	00110100	4	
053	35	00110101	5	
054	36	00110110	6	
055	37	00110111	7	
056	38	00111000	8	
057	39	00111001	9	
058	3A	00111010	:	Colon
059	3B	00111011	;	Semi-colon
060	3C	00111100	<	Less than sign
061	3D	00111101	=	Equal sign
062	3E	00111110	>	Greater than sign
063	3F	00111111	?	Question mark

DEC	HEX	BIN	CHAR	BEZEICHNUNG	DEC	HEX	BIN	CHAR	BEZEICHNUNG
064	40	01000000	@	AT symbol	096	60	01100000	`	
065	41	01000001	A		097	61	01100001	a	
066	42	01000010	B		098	62	01100010	b	
067	43	01000011	C		099	63	01100011	c	
068	44	01000100	D		100	64	01100100	d	
069	45	01000101	E		101	65	01100101	e	
070	46	01000110	F		102	66	01100110	f	
071	47	01000111	G		103	67	01100111	g	
072	48	01001000	H		104	68	01101000	h	
073	49	01001001	I		105	69	01101001	i	
074	4A	01001010	J		106	6A	01101010	j	
075	4B	01001011	K		107	6B	01101011	k	
076	4C	01001100	L		108	6C	01101100	l	
077	4D	01001101	M		109	6D	01101101	m	
078	4E	01001110	N		110	6E	01101110	n	
079	4F	01001111	O		111	6F	01101111	o	
080	50	01010000	P		112	70	01110000	p	
081	51	01010001	Q		113	71	01110001	q	
082	52	01010010	R		114	72	01110010	r	
083	53	01010011	S		115	73	01110011	s	
084	54	01010100	T		116	74	01110100	t	
085	55	01010101	U		117	75	01110101	u	
086	56	01010110	V		118	76	01110110	v	
087	57	01010111	W		119	77	01110111	w	
088	58	01011000	X		120	78	01111000	x	
089	59	01011001	Y		121	79	01111001	y	
090	5A	01011010	Z		122	7A	01111010	z	
091	5B	01011011	[	Left opening bracket	123	7B	01111011	{	Left opening brace
092	5C	01011100	\	Back slash	124	7C	01111100		Vertical bar
093	5D	01011101	]	Right closing bracket	125	7D	01111101	}	Right closing brace
094	5E	01011110	^	Caret circumflex	126	7E	01111110	~	Tilde
095	5F	01011111	_	Underscore	127	7F	01111111	DEL	Delete

Aussgangslage ist ASCII-7Bit erweitert um 1 Bit auf 8 Bit.
Zeichen 128 bis 255 gemäss dem ISO-8859-Standard (Zeichentabelle)

ISO-Standard 8859-1 = ANSI-ASCII (ANSI: American National Standards Institute)

ISO-Standard 8859-1 = Latin-1, Westeuropäisch oder ANSI-ASCII

ISO-Standard 8859-2 = Latin-2, Mitteleuropäisch

ISO-Standard 8859-3 = Latin-3, Südeuropäisch

ISO-Standard 8859-4 = Latin-4, Nordeuropäisch

ISO-Standard 8859-5 = Kyrillisch

ISO-Standard 8859-6 = Arabisch

ISO-Standard 8859-7 = Griechisch

ISO-Standard 8859-8 = Hebräisch

ISO-Standard 8859-9 = Latin-5, Türkisch

ISO-Standard 8859-10 = Latin-6, Nordisch

ISO-Standard 8859-11 = Thai

ISO-Standard 8859-12 = (existiert nicht)

ISO-Standard 8859-13 = Latin-7, Baltisch

ISO-Standard 8859-14 = Latin-8, Keltisch

ISO-Standard 8859-15 = Latin-9, Westeuropäisch

ISO-Standard 8859-16 = Latin-10, Südosteuropäisch

Unicode (ASCII-Nachfolger)

- Maximal **8 Byte** (=64 Bit)
U+xxxx'xxxx
- Unicode **V2.0** nutzt 1'114'112 verschiedene Zeichen
U+0000'0000 bis U+0010'FFFF

Unicode Transformations Formate

- **UTF-8** belegt pro Zeichen 1, 2, 3 oder 4 Byte
- **UTF-16** belegt pro Zeichen 2 oder 4 Byte
- **UTF** = **U**CS **T**ransformation **F**ormat, wobei **UCS** = **U**niversal-**C**oded-Character-**S**et

<u>Unicode-Bereich (Hexadez.)</u>	<u>UTF-8 Kodierung (Binär)</u>	<u>Bemerkungen</u>
0000 0000 – 0000 007F	0xxx xxxx	In diesem Bereich (128 Zeichen) entspricht UTF-8 genau dem ASCII-Code: Das höchste Bit ist 0, die restliche 7-Bit-Kombination ist das ASCII-Zeichen
0000 0080 – 0000 07FF	110xxxxx 10xxxxxx	Das erste Byte enthält binär 11xxxxxx, die folgenden Bytes 10xxxxxx; die x stehen für die fortlaufende Bitkombination des Unicode-Zeichens. Die Anzahl der Einsen vor der höchsten 0 im ersten Byte ist die Anzahl der Bytes für das Zeichen. (In Klammern jeweils die theoretisch maximal möglichen.)
0000 0800 – 0000 FFFF	1110xxxx 10xxxxxx 10xxxxxx	
0001 0000 – 0010 FFFF [0001 0000 – 001F FFFF]	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	

Zeichen	Unicode	Unicode (Binär)	UTF-8 (Binär)	UTF-8 (Hexadez.)
Buchstabe y	U+0079	00000000 0 1111001	01111001	0x79
Buchstabe ä	U+00E4	00000000 1110 0100	11000011 1010 0100	0xC3 0xA4
Zeichen für eingetragene Marke ®	U+00AE	0000 0000 1010 1110	11000010 1010 1110	0xC2 0xAE
Eurozeichen €	U+20AC	00 10 0000 1010 1100	11100010 10000010 10101100	0xE2 0x82 0xAC

Sie haben nun erfahren, wie UTF-8 funktioniert.

Das zweite, wichtige Unicode Transformationsformat ist UTF-16.

Erfahren sie nun hier im Selbststudium, wie UTF-16 funktioniert:

<https://de.wikipedia.org/wiki/UTF-16>

Beantworten sie nun die folgenden Kontrollfragen:

- Wo wird UTF8 eingesetzt, wo UTF-16?
- Was bedeutet die Byteorder bei UTF-16?
- Was bedeutet «abwärtskompatibel» im Zusammenhang mit UTF-8? Existiert diese «Abwärtskompatibilität» auch bei UTF-16?
- Wie beurteilen sie den Speicherbedarf von deutsch- oder englischsprachigem Text bei ISO-8859-Codierung, UTF-8-Codierung und UTF-16-Codierung?
- Wann wird ein UTF-16-Code vier Byte lang?
- Wie müssen sie vorgehen, wenn sie z.B. in Word ein Zeichen eingeben wollen, das sie auf der Tastatur zwar nicht finden, dessen Unicode sie aber kennen? (z.B. das Eurozeichen)
- Was wird das Problem sein, wenn sie auf dieselbe Weise wie in der vorangegangenen Aufgabe das Unicode-Zeichen des Violinschlüssels (Musiknoten) in eine Wordtext einfügen wollen?

Lösungen zu den Kontrollfragen:

- Wo wird UTF8 eingesetzt, wo UTF-16?
UTF8: Internetprotokolle, UTF16: .Net-Framework, Java, Tcl
- Was bedeutet die Byteorder bei UTF-16?
Big Endian (UTF-16BE), Little Endian (UTF-16LE)
*UTF16-BE-BOM: **FE FF**... UTF16-LE-BOM: **FF FE**... Siehe auch Beispiel im Anschluss!*
- Was bedeutet «abwärtskompatibel» im Zusammenhang mit UTF-8? Existiert diese «Abwärtskompatibilität» auch bei UTF-16?
Die ersten 128 Unicodezeichen (U+0000 bis U+007F entsprechen den Positionen 0–127 in allen ISO-8859-Varianten (7-Bit-ASCII). In UTF-8 deckungsgleich als ein Byte dargestellt.
Bei UTF16 existiert dies nicht.
- Wie beurteilen sie den Speicherbedarf von deutsch- oder englischsprachigem Text bei ISO-8859-Codierung, UTF-8-Codierung und UTF-16-Codierung?
ISO-8859-Codierung und UTF-8-Codierung ist effizienter, weil UTF16 immer 2Byte belegt.
- Wann wird ein UTF-16-Code vier Byte lang?
Unicode-Zeichen außerhalb der BMP (U+10000 bis U+10FFFF). BMP=Basic Multilingual Plane
- Wie müssen sie vorgehen, wenn sie z.B. in Word ein Zeichen eingeben wollen, das sie auf der Tastatur zwar nicht finden, dessen Unicode sie aber kennen? (z.B. das Eurozeichen)
U+20AC eingeben, danach ALT-C drücken.
- Was wird das Problem sein, wenn sie auf dieselbe Weise wie in der vorangegangenen Aufgabe das Unicode-Zeichen des Violinschlüssels (U+1D11E) in eine Wordtext einfügen wollen?
Gewählter Font-Satz wie z.B. Arial enthält Violinzeichen nicht.

Codevergleiche für das Wort «€URO»

Der ANSI-ASCII-Code für das €-Zeichen lautet 0x80 oder 1000'0000

Der Unicode für das €-Zeichen lautet U+20AC oder 0010'0000 1010'1100

€URO in ANSI-ASCII

€		U		R		O	
8	0	5	5	5	2	4	F
1000	0000	0101	0101	0101	0010	0100	1111

€URO in UTF-8

€						U		R		O	
E	2	8	2	A	C	5	5	5	2	4	F
1110	0010	1000	0010	1010	1100	0101	0101	0101	0010	0100	1111

€URO in UTF-16-BE

BE-16	€					U				R				O			
FE	FF	2	0	A	C	0	0	5	5	0	0	5	2	0	0	4	F
		0010	0000	1010	1100	0000	0000	0101	0101	0000	0000	0101	0010	0000	0000	0100	1111

€URO in UTF-16-LE

LE-16	€					U				R				O			
FF	FE	A	C	2	0	5	5	0	0	5	2	0	0	4	F	0	0
		1010	1100	0010	0000	0101	0101	0000	0000	0101	0010	0000	0000	0100	1111	0000	0000